C++ - Module 04

Polymorphisme de sous-type (Subtype Polymorphism), classes abstraites (Abstract Classes) et interfaces (Interfaces)

Résumé : ce document contient les exercices du Module 04 des modules C++.

Version : 13.1

Sommaire

I Introduction 2

II Règles générales 3

III Instructions sur l'IA (AI Instructions) 5

IV Exercice 00 : Polymorphism 7

V Exercice 01 : I don't want to set the world on fire 9

VI Exercice 02 : Abstract class 10

VII Exercice 03 : Interface & recap 11

VIII Rendu et évaluation par les pairs (Submission and Peer Evaluation) 15

Chapitre I

Introduction

C++ est un langage de programmation généraliste créé par Bjarne Stroustrup comme une extension du langage C, ou « C with Classes » (source : Wikipedia).

Le but de ces modules est de vous initier à la **Programmation Orientée Objet** (**Object-Oriented Programming**). Ce sera le point de départ de votre parcours en C++. De nombreux langages sont recommandés pour apprendre la POO, mais nous avons choisi le C++ car il dérive de votre vieil ami le C. Comme le C++ est un langage complexe, et pour garder les choses simples, votre code devra respecter le standard **C++98**.

Nous sommes conscients que le C++ moderne diffère beaucoup sur de nombreux aspects. Donc, si vous voulez devenir un développeur C++ compétent, c'est à vous de poursuivre votre parcours au-delà du Common Core de 42 !

Chapitre II

Règles générales

Compilation (Compiling)

- Compilez votre code avec `c++` et les flags `-Wall -Wextra -Werror`
- Votre code doit toujours compiler si vous ajoutez le flag `-std=c++98`

Formatage et conventions de nommage (Formatting and naming conventions)

- Les répertoires d'exercices seront nommés ainsi : `ex00, ex01, ... , exn`
- Nommez vos fichiers, classes, fonctions, fonctions membres (member functions) et attributs comme demandé dans les consignes.
- Écrivez les noms de classes en format **UpperCamelCase**. Les fichiers contenant le code d'une classe seront toujours nommés d'après le nom de la classe. Par exemple : `ClassName.hpp/ClassName.h`, `ClassName.cpp` ou `ClassName.tpp`. Ainsi, si vous avez un fichier header contenant la définition d'une classe « BrickWall » représentant un mur de briques, son nom sera `BrickWall.hpp`.
- Sauf indication contraire, chaque message affiché doit se terminer par un retour à la ligne et être affiché sur la sortie standard.
- *Goodbye Norminette !* Aucun style de code n'est imposé dans les modules C++. Vous pouvez suivre votre style préféré. Mais gardez en tête qu'un code que vos correcteurs (peer evaluators) ne comprennent pas est un code qu'ils ne peuvent pas noter. Faites de votre mieux pour écrire un code propre et lisible.

Autorisé / Interdit (Allowed/Forbidden)

Vous ne codez plus en C. C'est l'heure du C++ ! Donc :

- Vous pouvez utiliser presque tout de la bibliothèque standard. Ainsi, plutôt que de vous limiter à ce que vous connaissez déjà, il serait malin d'utiliser autant que possible les versions « C++-istes » des fonctions C que vous avez l'habitude d'utiliser.
- Cependant, vous ne pouvez utiliser aucune autre bibliothèque externe. Cela veut dire que le C++11 (et ses formes dérivées) et les bibliothèques **Boost** sont interdits. Les fonctions suivantes sont aussi interdites : `*printf()`, `*alloc()` et `free()`. Si vous les utilisez, votre note sera 0, et c'est tout.
- Notez que, sauf indication explicite, les mots-clés `using namespace <ns_name>` et `friend` sont interdits. Sinon, votre note sera -42.
- **Vous êtes autorisé à utiliser la STL uniquement dans les Modules 08 et 09.** Cela veut dire : pas de **Containers** (vector/list/map, etc.) et pas de **Algorithms** (tout ce qui nécessite d'inclure le

header `<algorithm>`) d'ici là. Sinon, votre note sera -42.

Quelques contraintes de conception (A few design requirements)

- Les fuites mémoire (**memory leaks**) existent aussi en C++. Quand vous allouez de la mémoire (avec le mot-clé `new`), vous devez éviter les fuites.
- Du Module 02 au Module 09, vos classes doivent être conçues selon la **Orthodox Canonical Form**, sauf indication contraire explicite.
- Toute implémentation de fonction placée dans un fichier header (sauf pour les function templates) vaut 0 à l'exercice.
- Vous devez pouvoir utiliser chacun de vos headers indépendamment des autres. Ils doivent donc inclure toutes les dépendances dont ils ont besoin. Cependant, vous devez éviter le problème de la double inclusion en ajoutant des **include guards**. Sinon, votre note sera 0.

À lire (Read me)

- Vous pouvez ajouter des fichiers supplémentaires si besoin (par exemple pour découper votre code). Comme ces exercices ne sont pas vérifiés par un programme, vous pouvez le faire tant que vous rendez les fichiers obligatoires.
- Parfois, les consignes d'un exercice semblent courtes, mais les exemples peuvent montrer des exigences qui ne sont pas écrites explicitement dans les instructions.
- Lisez chaque module en entier avant de commencer ! Vraiment, faites-le.
- Par Odin, par Thor ! Utilisez votre cerveau !!!

ATTENTION
Concernant le Makefile pour les projets C++, les mêmes règles qu'en C s'appliquent (voir le chapitre de la Norme sur le Makefile).

ASTUCE
Vous allez devoir implémenter beaucoup de classes. Cela peut sembler fastidieux, sauf si vous savez scripter votre éditeur de texte préféré.

INFO
On vous laisse une certaine liberté pour réaliser les exercices. Cependant, suivez les règles obligatoires et ne soyez pas paresseux. Vous passeriez à côté de beaucoup d'informations utiles ! N'hésitez pas à vous documenter sur les concepts théoriques.

Chapitre III

Instructions sur l’IA

● Contexte

Ce projet est conçu pour vous aider à découvrir les briques fondamentales de votre formation à 42.

Pour bien ancrer les connaissances et compétences clés, il est essentiel d’adopter une approche réfléchie de l’utilisation des outils d’IA et de leur support.

Un véritable apprentissage des fondamentaux demande un vrai effort intellectuel — à travers le défi, la répétition et les échanges d’apprentissage entre pairs (peer-learning).

Pour une vue plus complète de notre position sur l’IA — comme outil d’apprentissage, comme partie de la formation 42 et comme attente du marché du travail — merci de consulter la FAQ dédiée sur l’intranet.

● Message principal

☛ Construisez des bases solides, sans raccourcis.

☛ Développez vraiment vos compétences techniques (tech) et transversales (power skills).

☛ Vivez un vrai peer-learning, commencez à apprendre à apprendre et à résoudre de nouveaux problèmes.

☛ Le parcours d’apprentissage est plus important que le résultat.

☛ Informez-vous sur les risques associés à l’IA, et développez de bonnes pratiques de contrôle et des contre-mesures pour éviter les pièges courants.

● Règles de l’apprenant (Learner rules)

- Vous devez raisonner sur les tâches qui vous sont confiées, surtout avant de vous tourner vers l’IA.
- Vous ne devez pas demander de réponses directes à l’IA.
- Vous devez vous informer sur l’approche globale de 42 concernant l’IA.

● Résultats de la phase (Phase outcomes)

Dans cette phase fondamentale, vous obtiendrez les résultats suivants :

- Acquérir de vraies bases techniques et de code.
- Savoir pourquoi et comment l’IA peut être dangereuse pendant cette phase.

● Commentaires et exemple (Comments and example)

- Oui, nous savons que l'IA existe — et oui, elle peut résoudre vos projets. Mais vous êtes ici pour apprendre, pas pour prouver que l'IA a appris. Ne perdez pas votre temps (ni le nôtre) juste pour démontrer que l'IA sait résoudre le problème donné.
- Apprendre à 42, ce n'est pas connaître la réponse — c'est développer la capacité d'en trouver une. L'IA vous donne directement la réponse, mais cela vous empêche de construire votre propre raisonnement. Et le raisonnement demande du temps, des efforts et passe par l'échec. Le chemin vers la réussite n'est pas censé être facile.
- Gardez en tête que pendant les examens, l'IA n'est pas disponible — pas d'internet, pas de smartphone, etc. Vous verrez vite si vous vous êtes trop reposé sur l'IA dans votre apprentissage.
- Le peer-learning vous expose à d'autres idées et approches, améliore vos compétences interpersonnelles et votre capacité à penser de façon divergente. C'est bien plus précieux que de simplement discuter avec un bot. Alors ne soyez pas timide — parlez, posez des questions et apprenez ensemble !
- Oui, l'IA fera partie du cursus — à la fois comme outil d'apprentissage et comme sujet en soi. Vous aurez même l'occasion de construire votre propre logiciel d'IA. Pour en savoir plus sur notre approche progressive (crescendo approach), consultez la documentation disponible sur l'intranet.

✓ Bonne pratique (Good practice) :

Je suis bloquée sur une nouvelle notion. Je demande à quelqu'un à côté comment il l'a abordée. On discute pendant 10 minutes — et d'un coup, ça fait « tilt ». Je comprends.

✗ Mauvaise pratique (Bad practice) :

J'utilise l'IA en cachette, je copie du code qui a l'air correct. Pendant la peer evaluation, je ne sais rien expliquer. J'échoue. À l'examen — pas d'IA — je suis de nouveau bloquée. J'échoue.

Chapitre IV

Exercice 00 : Polymorphism

Exercice : 00
Polymorphism
Répertoire : *ex00/*
Fichiers à rendre : `Makefile, main.cpp, *.cpp, *.{h, hpp}`
Interdit : `None`

Pour chaque exercice, vous devez fournir les **tests les plus complets** possibles. Les constructeurs et destructeurs de chaque classe doivent afficher des messages spécifiques. N'utilisez pas le même message pour toutes les classes.

Commencez par implémenter une classe de base simple appelée **Animal**. Elle a un attribut protected :

- `std::string type;`

Implémentez une classe **Dog** qui hérite de Animal.
Implémentez une classe **Cat** qui hérite de Animal.

Ces deux classes dérivées doivent initialiser leur champ `type` selon leur nom. Ainsi, le type de Dog sera initialisé à `"Dog"`, et le type de Cat sera initialisé à `"Cat"`. Le type de la classe Animal peut être laissé vide ou mis à la valeur de votre choix.

Tout animal doit pouvoir utiliser la fonction membre :
`makeSound()`
Elle affichera un son approprié (les chats n'aboient pas).

Exécuter ce code doit afficher les sons spécifiques des classes Dog et Cat, et non celui de Animal.

```
int main()
{
    const Animal* meta = new Animal();
    const Animal* j = new Dog();
    const Animal* i = new Cat();

    std::cout << j->getType() << " " << std::endl;
    std::cout << i->getType() << " " << std::endl;
    i->makeSound(); //will output the cat sound!
    j->makeSound();
    meta->makeSound();

    ...

    return 0;
}
```

Pour vous assurer d’avoir compris comment cela fonctionne, implémentez une classe **WrongCat** qui hérite d’une classe **WrongAnimal**. Si vous remplacez Animal et Cat par les « wrong » dans le code ci-dessus, le WrongCat doit afficher le son de WrongAnimal.

Implémentez et rendez davantage de tests que ceux donnés ci-dessus.

Chapitre V

Exercice 01 : I don’t want to set the world on fire

Exercice : 01
I don’t want to set the world on fire
Répertoire : *ex01/*
Fichiers à rendre : `Files from previous exercise + *.cpp, *.{h, hpp}`
Interdit : `None`

Les constructeurs et destructeurs de chaque classe doivent afficher des messages spécifiques.

Implémentez une classe **Brain**. Elle contient un tableau de 100 `std::string` appelé `ideas`.
Ainsi, Dog et Cat auront un attribut privé `Brain*`.
À la construction, Dog et Cat créeront leur Brain avec `new Brain();`
À la destruction, Dog et Cat feront un `delete` de leur Brain.

Dans votre fonction main, créez et remplissez un tableau d’objets **Animal**. La moitié seront des objets **Dog** et l’autre moitié des objets **Cat**. À la fin de l’exécution de votre programme, parcourez ce tableau en boucle et faites un delete de chaque Animal. Vous devez supprimer directement les dogs et les cats en tant qu’Animals. Les destructeurs appropriés doivent être appelés dans l’ordre attendu.

N’oubliez pas de vérifier les **memory leaks**.

Une copie d’un Dog ou d’un Cat ne doit pas être superficielle (shallow). Vous devez donc tester que vos copies sont des copies profondes (**deep copies**) !

```
int main()
{
    const Animal* j = new Dog();
    const Animal* i = new Cat();


    delete j;//should not create a leak
    delete i;
    ...

    return 0;
}
```

Implémentez et rendez davantage de tests que ceux donnés ci-dessus.

Chapitre VI

Exercice 02 : Abstract class

Exercice : 02
Abstract class
Répertoire : *ex02/*
Fichiers à rendre : `Files from previous exercise + *.cpp, *.{h, hpp}`
Interdit : `None`

Créer des objets Animal n’a finalement pas de sens. C’est vrai, ils ne font aucun son !

Pour éviter toute erreur possible, la classe Animal par défaut ne doit pas être instanciable. Corrigez la classe Animal pour que personne ne puisse l’instancier. Tout doit fonctionner comme avant.

Si vous le souhaitez, vous pouvez changer le nom de la classe en ajoutant un préfixe **A** à Animal.

Chapitre VII

Exercice 03 : Interface & recap

Exercice : 03
Interface & recap
Répertoire : *ex03/*
Fichiers à rendre : `Makefile, main.cpp, *.cpp, *.{h, hpp}`
Interdit : `None`

Les interfaces n'existent pas en C++98 (ni même en C++20). Cependant, les classes abstraites pures (**pure abstract classes**) sont couramment appelées interfaces. Donc, dans ce dernier exercice, essayons d'implémenter des interfaces afin de vérifier que vous avez bien compris ce module.

Complétez la définition de la classe **AMateria** suivante et implémentez les fonctions membres nécessaires.

```
class AMateria
{
   protected:
           [...]

   public:
           AMateria(std::string const & type);
           [...]

           std::string const & getType() const; //Returns the materia type

           virtual AMateria* clone() const = 0;
           virtual void use(ICharacter& target);
};
```

Implémentez les classes concrètes pour les Materias : **Ice** et **Cure**. Utilisez leurs noms en minuscules ("ice" pour Ice, "cure" pour Cure) pour définir leurs types. Bien sûr, leur fonction membre `clone()` renverra une nouvelle instance du même type (c'est-à-dire que si vous clonez une Materia Ice, vous obtiendrez une nouvelle Materia Ice).

La fonction membre `use(ICharacter&)` affichera :

- Ice : `"* shoots an ice bolt at <name> *"`
- Cure : `"* heals <name>'s wounds *"`

`<name>` est le nom du Character passé en paramètre. N'affichez pas les chevrons (< et >).

ASTUCE
Lors de l'assignation d'une Materia à une autre, copier le type n'a pas de sens.

Écrivez la classe concrète **Character** qui implémentera l'interface suivante :

```
class ICharacter
{
   public:
           virtual ~ICharacter() {}
           virtual std::string const & getName() const = 0;
           virtual void equip(AMateria* m) = 0;
           virtual void unequip(int idx) = 0;
           virtual void use(int idx, ICharacter& target) = 0;
};
```

Le **Character** possède un inventaire de 4 emplacements (slots), ce qui signifie au maximum 4 Materias. L'inventaire est vide à la construction. Il équipe les Materias dans le premier emplacement vide qu'il trouve, dans l'ordre suivant : du slot 0 au slot 3. S'il essaie d'ajouter une Materia à un inventaire plein, ou d'utiliser/déséquiper une Materia inexistante, il ne doit rien se passer (mais les bugs restent interdits). La fonction membre `unequip()` ne doit PAS supprimer la Materia !

ASTUCE
Gérez comme vous voulez les Materias que votre personnage laisse par terre. Sauvegardez les adresses avant d'appeler unequip(), ou autre chose, mais n'oubliez pas que vous devez éviter les memory leaks.

La fonction membre `use(int, ICharacter&)` devra utiliser la Materia du slot[idx], et passer le paramètre target à la fonction `AMateria::use`.

ATTENTION
L'inventaire de votre personnage devra pouvoir supporter n'importe quel type d'AMateria.

Votre **Character** doit avoir un constructeur prenant son nom en paramètre. Toute copie (avec le copy constructor ou le copy assignment operator) d'un Character doit être **deep** (profonde). Pendant la copie, les Materias d'un Character doivent être supprimées avant que les nouvelles soient ajoutées à son inventaire. Bien sûr, les Materias doivent être supprimées quand un Character est détruit.

Écrivez la classe concrète **MateriaSource** qui implémentera l'interface suivante :

```
class IMateriaSource
{
   public:
           virtual ~IMateriaSource() {}
           virtual void learnMateria(AMateria*) = 0;
           virtual AMateria* createMateria(std::string const & type) = 0;
};
```

- `learnMateria(AMateria*)`
  Copie la Materia passée en paramètre et la stocke en mémoire pour qu'elle puisse être clonée plus tard. Comme le Character, la **MateriaSource** peut connaître au maximum 4 Materias. Elles ne sont pas forcément uniques.
- `createMateria(std::string const &)`
  Renvoie une nouvelle Materia. Celle-ci est une copie de la Materia précédemment apprise par la **MateriaSource** dont le type est égal à celui passé en paramètre. Renvoie 0 si le type est inconnu.

En résumé, votre **MateriaSource** doit pouvoir apprendre des « modèles » (templates) de Materias pour les créer au besoin. Ensuite, vous pourrez générer une nouvelle Materia en utilisant simplement une string qui identifie son type.

Exécuter ce code :

```
int main()
{
    IMateriaSource* src = new MateriaSource();
    src->learnMateria(new Ice());
    src->learnMateria(new Cure());

    ICharacter* me = new Character("me");

    AMateria* tmp;
    tmp = src->createMateria("ice");
    me->equip(tmp);
    tmp = src->createMateria("cure");
    me->equip(tmp);

    ICharacter* bob = new Character("bob");

    me->use(0, *bob);
    me->use(1, *bob);

    delete bob;
    delete me;
    delete src;

    return 0;
}
```

Doit afficher :

Terminal

```
$> clang++ -W -Wall -Werror *.cpp
$> ./a.out | cat -e
* shoots an ice bolt at bob *$
* heals bob's wounds *$
```

Comme d’habitude, implémentez et rendez davantage de tests que ceux donnés ci-dessus.

INFO
Vous pouvez valider ce module sans faire l’exercice 03.

Chapitre VIII

Rendu et évaluation par les pairs

Rendez votre travail dans votre dépôt `Git` comme d'habitude. Seul le travail présent dans votre dépôt sera évalué pendant la soutenance. N'hésitez pas à vérifier deux fois les noms de vos dossiers et fichiers pour vous assurer qu'ils sont corrects.

Pendant l'évaluation, une brève **modification du projet** peut parfois être demandée. Il peut s'agir d'un petit changement de comportement, de quelques lignes de code à écrire ou réécrire, ou d'une fonctionnalité facile à ajouter.

Cette étape peut **ne pas s'appliquer à tous les projets**, mais vous devez y être préparé si elle est mentionnée dans les consignes d'évaluation (evaluation guidelines).

Cette étape sert à vérifier votre compréhension réelle d'une partie précise du projet. La modification peut être réalisée dans n'importe quel environnement de développement de votre choix (par exemple votre configuration habituelle), et elle doit être faisable en quelques minutes — sauf si un délai précis est défini dans l'évaluation.
Vous pouvez par exemple devoir faire une petite mise à jour d'une fonction ou d'un script, modifier un affichage, ou adapter une structure de données pour stocker de nouvelles informations, etc.

Les détails (portée, cible, etc.) seront précisés dans les **consignes d'évaluation** et peuvent varier d'une évaluation à l'autre pour un même projet.